

SIMATS ENGINEERING CO2-ASSESSMENT TOOL-1

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 Total Marks:25 Duration : 1 Hour

Application Based Question

Code of Conduct:

I GIRIJA B/ 192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Girija B

SIMATS ENGINEERING ASSESSMENT TOOL

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a Deterministic Finite Automaton (DFA) over the alphabet $\Sigma = \{0,1\}$ to validate passwords that end with the pattern "01". Further, minimize the DFA and analyze how the optimized design improves system efficiency in real-world hardware implementations

Alphabet: $\Sigma = \{0,1\}$

Language Accepted: All binary strings ending with "01".

States

- **q0** – Initial state
- **q1** – Last symbol read is 0
- **q2** – Accept state (string ends with 01)

Transition Table

Present State	Input 0	Input 1
→ q0	q1	q0
q1	q1	q2
*q2	q1	q0

(*q2 is the final/accept state.)

Minimization

Using the partition method:

- Final state: {q2}
- Non-final states: {q0, q1}
- Since q0 and q1 have different transitions on input 1, they cannot be merged.

Therefore, the DFA is already minimized.

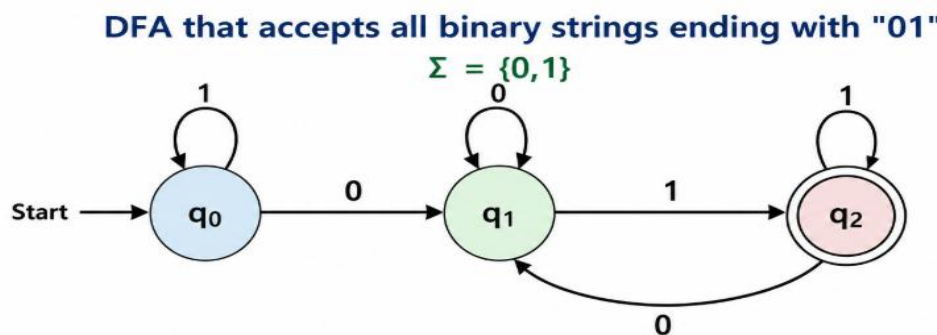
- **Original DFA:** 3 states
- **Minimized DFA:** 3 states

Efficiency Analysis

- Uses only **3 states**, requiring less memory.
- Processes one input symbol at a time with **$O(n)$** time complexity.
- Deterministic behavior ensures fast and reliable password validation.
- Fewer states reduce hardware complexity, power consumption, and implementation cost.
- Suitable for **embedded systems, ATM machines, smart cards, and authentication modules.**

Conclusion

The minimized DFA accepts all binary strings ending with "**01**". Since no states can be merged, the original DFA is already minimal, making it efficient for real-time password validation in digital security systems.



2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$. Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

Regular Expression for the Pattern "aaaabb"

Given:

- Alphabet: $\Sigma = \{a, b\}$
- Required Pattern: "aaaabb"

Regular Expression

`aaaabb`

This regular expression accepts only one string, namely "aaaabb", and rejects every other string.

Examples

Input String	Result
aaaabb	☒ Accepted
aaabb	☒ Rejected
aaaab	☒ Rejected
aaaabbb	☒ Rejected
bbbbaa	☒ Rejected

Use of Regular Languages and Regular Expressions in Web Applications

Regular languages and regular expressions are widely used to validate user inputs efficiently before processing them.

Applications

- Username validation
- Password format checking
- Email address validation
- Phone number verification
- PIN and OTP validation
- ZIP/Postal code validation
- Date and URL validation

Advantages

- Fast Validation: Matches input in linear time $O(n)$.
- Improves Security: Prevents invalid or malicious input.
- Reduces Server Load: Filters incorrect inputs on the client side.
- Easy to Implement: Supported in JavaScript, Python, Java, PHP, and other programming languages.
- Improves User Experience: Provides instant feedback for invalid inputs.

Conclusion

The regular expression `aaaabb` accepts only the string "aaaabb" over the alphabet $\Sigma = \{a, b\}$. Regular expressions are an efficient way to validate user inputs in web applications, ensuring correctness, enhancing security, and improving overall system performance.

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression `a*b*` can be converted into an equivalent finite automaton (FA). Further, explain why regular expressions and finite automata are equivalent models and how they are effectively

used in real-time detection systems.

Given

- **Regular Expression:** a^*b^*
- **Alphabet:** $\Sigma = \{a, b\}$

The regular expression a^*b^* represents **zero or more 'a's followed by zero or more 'b's**. Once a 'b' appears, no 'a' is allowed afterward.

Examples Accepted

- ϵ (empty string)
- a
- aa
- b
- bb
- ab

Examples Rejected

- ba
- aba
- baa
- abab

Equivalent Finite Automaton (FA)

States

- **q0** – Initial and accepting state (reading a's)
- **q1** – Accepting state (reading b's)
- **q2** – Dead/Trap state

Transition Table

Present State	Input a	Input b
→ *q0	q0	q1
*q1	q2	q1
q2	q2	q2

Equivalence of Regular Expressions and Finite Automata

Regular expressions and finite automata are **equivalent models** because:

- Every regular expression can be converted into an equivalent finite automaton.
- Every finite automaton can be represented by an equivalent regular expression.
- Both recognize exactly the **regular languages**.

This equivalence is established by **Kleene's Theorem**.

Applications in Real-Time Detection Systems

Regular expressions and finite automata are widely used in:

- Intrusion Detection Systems (IDS)

- Compiler lexical analysis
- Malware and virus detection
- Network packet filtering
- Log file monitoring
- Search engines and text editors
- Input validation in web applications

Advantages

- **Fast pattern matching** with $O(n)$ time complexity.
- **Low memory usage** due to finite states.
- **Deterministic execution** ensures reliable results.
- Suitable for **real-time processing** of large data streams.
- Easy to implement in software and hardware.

Conclusion

The regular expression a^*b^* is converted into an equivalent finite automaton with **three states (including a trap state)**. Regular expressions and finite automata are equivalent computational models capable of recognizing the same class of regular languages. Their efficiency, speed, and simplicity make them ideal for real-time pattern detection in intrusion detection systems, compilers, and monitoring applications.

4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L = \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

Given Language

$$L = \{a^n b^n \mid n \geq 0\}$$

This language contains strings with an **equal number of a's followed by an equal number of b's**.

Examples:

ϵ , ab, aabb, aaabbb, aaaabbbb

Pumping Lemma Proof

Assume that **L is regular**.

According to the **Pumping Lemma**, there exists a pumping length **p** such that every string **w** in **L** with $|w| \geq p$ can be written as:

$$w = xyz$$

where:

1. $|xy| \leq p$
2. $|y| > 0$
3. $xy^i z \in L$ for all $i \geq 0$

Choose a String

Let

$$w = a^p b^p$$

Since $|xy| \leq p$, the substring **y** consists only of **a's**.

Suppose

$$y = a^k, k > 0$$

Now pump **y** with $i = 2$:

$$xy^2z = a^{p+k}b^p$$

The resulting string has **more a's than b's**, so

$$a^{p+k}b^p \notin L$$

This contradicts the Pumping Lemma, which states that the pumped string must also belong to **L**.

Hence, our assumption is false.

Therefore,

$L = \{a^n b^n \mid n \geq 0\}$ is not a regular language.
--

Hence, **no finite automaton can recognize this language.**

Implications in Modern Compiler Design

Since finite automata cannot recognize languages such as $a^n b^n$, compilers use more powerful models for syntax analysis.

Applications

- Lexical Analysis → Uses **Finite Automata (FA)** to recognize identifiers, keywords, numbers, and operators.
- Syntax Analysis (Parsing) → Uses **Context-Free Grammars (CFGs)** and **Pushdown Automata (PDA)** to recognize nested and balanced structures.

Examples Requiring a PDA

- Balanced parentheses ()

- Matching { } and []
- Nested if-else statements
- Function call nesting
- Equal numbers of opening and closing symbols

Conclusion

The Pumping Lemma proves that the language

$$L = \{a^n b^n \mid n \geq 0\}$$

is **not regular** because pumping changes the number of **a's** without changing the number of **b's**, violating the language definition. Therefore, **finite automata cannot recognize this language**. In modern compilers, finite automata are used for **lexical analysis**, while **context-free grammars and pushdown automata** are required for **syntax analysis** to handle nested and balanced language constructs.

5.Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the closure properties of regular languages enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations remains regular, ensuring efficient implementation using finite automata

Closure Properties of Regular Languages

A language is **regular** if it can be recognized by a **Finite Automaton (FA)** or represented using a **Regular Expression (RE)**.

Regular languages are **closed** under several operations, meaning that applying these operations to regular languages always produces another regular language.

The important closure properties are:

- **Union ($\cup$)**
- **Concatenation ($\cdot$)**
- **Kleene Star ($*$)**
- **Intersection ($\cap$)**
- **Complement ($'$)**
- **Difference ($-$)**

Combining Multiple Detection Rules

In modern systems, multiple pattern-matching rules can be combined using closure properties.

Examples

- **Intrusion Detection Systems (IDS):**
 - Rule 1: Detect malicious login attempts.
 - Rule 2: Detect SQL injection patterns.
 - Combined using **Union** ($L_1 \cup L_2$).
- **Spam Filtering:**
 - Detect suspicious keywords.
 - Detect malicious URLs.
 - Detect phishing email formats.
 - Combined using **Union** and **Concatenation**.
- **Input Validation:**
 - Validate usernames.
 - Validate passwords.
 - Validate email addresses.
 - Combined using **Regular Expressions** and **Closure Operations**.

Why the Resulting Language Remains Regular

If L_1 and L_2 are regular languages, then:

- **Union:** $L_1 \cup L_2$ is regular.
- **Concatenation:** $L_1 L_2$ is regular.
- **Kleene Star:** L_1^* is regular.
- **Intersection:** $L_1 \cap L_2$ is regular.
- **Complement:** $\bar{L}_1$ is regular.
- **Difference:** $L_1 - L_2$ is regular.

Thus, combining multiple detection rules always produces another **regular language**, which can still be recognized by a **Finite Automaton**.

Advantages

- Enables combining multiple security rules efficiently.
- Fast pattern matching with **$O(n)$** time complexity.
- Requires constant memory for DFA implementation.
- Easy to implement in hardware and software.
- Provides reliable and deterministic detection.

Applications

- Intrusion Detection Systems (IDS)
- Spam Email Filtering
- Firewall Rule Matching
- Web Input Validation
- Compiler Lexical Analysis
- Malware Detection
- Network Packet Inspection

Conclusion

The closure properties of regular languages allow multiple detection rules to be combined using operations such as **union**, **concatenation**, **Kleene star**, **intersection**, **complement**, and **difference**. Since regular languages remain **regular** after these operations, the resulting language can still be recognized efficiently by **finite automata**. This ensures fast, reliable, and scalable pattern matching in modern systems such as intrusion detection, spam filtering, and input validation.